33. Search in Rotated Sorted Array Medium

Acceptance: 43.9% | LeetCode

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, *or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
**Input:** nums = [4,5,6,7,0,1,2], target = 0
**Output:** 4
```

Example 2:

```
**Input:** nums = [4,5,6,7,0,1,2], target = 3
**Output:** -1
```

Example 3:

```
**Input:** nums = [1], target = 0
**Output:** -1
```

Constraints:

- `1 <= nums.length <= 5000`
- `-104 <= nums[i] <= 104`
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- `-104 <= target <= 104`

Claude Opus Solution

Search in Rotated Sorted Array

1. Intuition

The core challenge: we need `O(log n)` search, which screams **binary search**. But the array isn't fully sorted—it's rotated.

The key insight: When you split a rotated sorted array at any midpoint, **at least one of the two halves is guaranteed to be properly sorted.**

Why? A rotation creates exactly one "break point" (where a larger number is followed by a smaller one). When we pick a `mid`, this break point can only fall on one side. The other side must be perfectly sorted.

```
[4, 5, 6, 7, 0, 1, 2]
          ^mid
Left half [4,5,6,7] is sorted, right half [0,1,2] is sorted (break is exactly at mid)
```

Once we know **which half is sorted**, we can use simple range comparisons to decide whether `target` lies within that sorted half. If yes, search there; if no, search the other half.

---

2. Approach

```
1. Set left = 0, right = n - 1
2. While left <= right:
   a. Compute mid = left + (right - left) // 2
   b. If nums[mid] == target, return mid

   c. Determine which half is sorted by comparing nums[mid] with nums[left]:

      CASE A: Left half [left..mid] is sorted (nums[mid] >= nums[left])
         - If target lies in this sorted range [nums[left], nums[mid]):
             search left  -> right = mid - 1
         - Else: search right -> left = mid + 1

      CASE B: Right half [mid..right] is sorted (nums[mid] < nums[left])
         - If target lies in this sorted range (nums[mid], nums[right]]:
             search right -> left = mid + 1
         - Else: search left -> right = mid - 1

3. Return -1 (not found)
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def search(self, nums: List[int], target: int) -> int:
        # Standard two-pointer binary search boundaries
        left, right = 0, len(nums) - 1

        while left <= right:
            # Compute mid this way to avoid integer overflow in other languages
            # (not an issue in Python, but a good habit)
            mid = left + (right - left) // 2

            # Direct hit: found the target
            if nums[mid] == target:
                return mid

            # KEY STEP: Figure out which half is sorted.
            # If nums[mid] >= nums[left], the LEFT half [left..mid] is sorted.
            # (Using >= handles the edge case when left == mid)
            if nums[mid] >= nums[left]:
                # Left half is sorted. Check if target falls inside it.
                # We use [nums[left], nums[mid]) — exclusive on mid since
                # we already checked nums[mid] != target above.
                if nums[left] <= target < nums[mid]:
                    # Target is within the sorted left half -> discard right
                    right = mid - 1
                else:
                    # Target must be in the unsorted right half
                    left = mid + 1

            # Otherwise, the RIGHT half [mid..right] is sorted.
            else:
                # Right half is sorted. Check if target falls inside it.
                # Range (nums[mid], nums[right]] — exclusive on mid.
                if nums[mid] < target <= nums[right]:
                    # Target is within the sorted right half -> discard left
                    left = mid + 1
                else:
                    # Target must be in the unsorted left half
                    right = mid - 1

        # Exhausted search space without finding target
        return -1
```

Walkthrough Example

`nums = [4,5,6,7,0,1,2]`, `target = 0`

left	right	mid	nums[mid]	Sorted half	Decision
0	6	3	7	Left `[4..7]` sorted	`0` not in `[4,7)` → go right, `left=4`
4	6	5	1	Left `[0,1]` sorted	`0` in `[0,1)` → go left, `right=4`
4	4	4	0	—	`nums[4]==0` → **return 4** ✓

4. Complexity Analysis

Metric	Complexity	Explanation
Time	`O(log n)`	Each iteration halves the search space, just like standard binary search. The extra "which half is sorted" check is `O(1)`.
Space	`O(1)`	Only a constant number of pointer variables are used; no recursion or extra data structures.

5. Alternative Approaches

Approach 1: Find pivot, then binary search (two-pass) - First, do a binary search to locate the rotation point (smallest element / pivot index `k`). - Then determine which sorted segment `target` belongs to and run a standard binary search on that segment. - Still `O(log n)`, but requires two passes—slightly more code, conceptually cleaner for some.

```python
def search(nums, target):
    n = len(nums)
    # Step 1: find pivot (index of minimum)
    lo, hi = 0, n - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]:
            lo = mid + 1
        else:
            hi = mid
    pivot = lo
    # Step 2: pick the correct sorted half, then standard binary search
    if pivot == 0 or target >= nums[0]:
        lo, hi = 0, pivot - 1 if pivot else n - 1
    else:
        lo, hi = pivot, n - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Approach 2: Linear scan — `O(n)`. Trivial but **violates** the `O(log n)` requirement; only acceptable if constraints were tiny.

Recommendation: The single-pass solution shown in Section 3 is the cleanest and most commonly expected in interviews—it elegantly handles rotation in one binary search loop. The key pattern to remember: **"one half is always sorted; check if target is in the sorted half."**

Codex Solution

1. Intuition

A rotated sorted array is made of **two sorted parts**.

Example:

```
nums = [4, 5, 6, 7, 0, 1, 2]
        -----------  --------
        sorted left  sorted right
```

Even though the whole array is not fully sorted, for any middle index `mid`, **at least one side must be sorted**:

- Either `nums[left] <= nums[mid]`, meaning the left half is sorted
- Or `nums[mid] <= nums[right]`, meaning the right half is sorted

Because all values are distinct, we can reliably determine which half is sorted.

Once we know which half is sorted, we check whether `target` lies inside that sorted half. If it does, we search there. Otherwise, we search the other half.

This allows us to discard half of the search space each time, just like normal binary search.

---

2. Approach

We use binary search with extra logic.

Pseudocode

```
left = 0
right = len(nums) - 1

while left <= right:
    mid = middle index between left and right
```

```
    if nums[mid] == target:
        return mid

    if nums[left] <= nums[mid]:
        # Left half is sorted

        if nums[left] <= target < nums[mid]:
            # Target is inside the sorted left half
            right = mid - 1
        else:
            # Target must be in the right half
            left = mid + 1

    else:
        # Right half is sorted

        if nums[mid] < target <= nums[right]:
            # Target is inside the sorted right half
            left = mid + 1
        else:
            # Target must be in the left half
            right = mid - 1

return -1
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def search(self, nums: List[int], target: int) -> int:
        # Standard binary search boundaries
        left = 0
        right = len(nums) - 1

        while left <= right:
            # Compute middle index.
            # This form avoids integer overflow in languages like Java/C++.
            # In Python, overflow is not an issue, but this is still a good habit.
            mid = left + (right - left) // 2

            # If we found the target, return its index immediately.
            if nums[mid] == target:
                return mid

            # Case 1:
            # The left half nums[left ... mid] is sorted.
            #
            # Because values are distinct, nums[left] <= nums[mid]
            # means everything from left to mid is in increasing order.
            if nums[left] <= nums[mid]:

                # Check if target lies inside this sorted left half.
                #
                # We use:
                # nums[left] <= target < nums[mid]
                #
                # The right side is strict because nums[mid] was already
                # checked above and is not equal to target.
                if nums[left] <= target < nums[mid]:
                    # Target is in the left sorted half.
                    right = mid - 1
                else:
                    # Target is not in the left sorted half,
                    # so it must be in the right half if it exists.
                    left = mid + 1

            # Case 2:
            # The right half nums[mid ... right] is sorted.
```

```
            else:

                # Check if target lies inside this sorted right half.
                #
                # We use:
                # nums[mid] < target <= nums[right]
                #
                # The left side is strict because nums[mid] was already
                # checked above and is not equal to target.
                if nums[mid] < target <= nums[right]:
                    # Target is in the right sorted half.
                    left = mid + 1
                else:
                    # Target is not in the right sorted half,
                    # so it must be in the left half if it exists.
                    right = mid - 1

        # If the loop finishes, target was not found.
        return -1
```

---

4. Complexity Analysis

Time Complexity: `O(log n)`

At every step, we discard half of the current search range, just like regular binary search.

So the number of iterations is proportional to:

```
log₂(n)
```

Therefore, the time complexity is:

```
O(log n)
```

Space Complexity: `O(1)`

We only use a few variables:

```
left, right, mid
```

No extra data structures are used.

Therefore, the space complexity is:

```
O(1)
```

---

5. Alternative Approaches

Alternative 1: Find Pivot First, Then Binary Search

You can solve the problem in two steps:

1. Find the index of the smallest element, also called the rotation pivot.
2. Decide whether to binary search in the left sorted part or right sorted part.

Example:

```
nums = [4, 5, 6, 7, 0, 1, 2]
                   ^
                   pivot index = 4
```

Then:

- Left sorted part: `[4, 5, 6, 7]`
- Right sorted part: `[0, 1, 2]`

This also runs in:

```
O(log n)
```

However, the one-pass approach above is usually cleaner because it searches directly without explicitly finding the pivot.

Alternative 2: Convert Indices Virtually

Another advanced method is to find the pivot and then perform binary search using a "virtual index" mapping.

This treats the rotated array as if it were sorted by translating indices:

```
real_index = (virtual_index + pivot) % n
```

This is elegant but slightly harder to understand than the direct one-pass binary search.